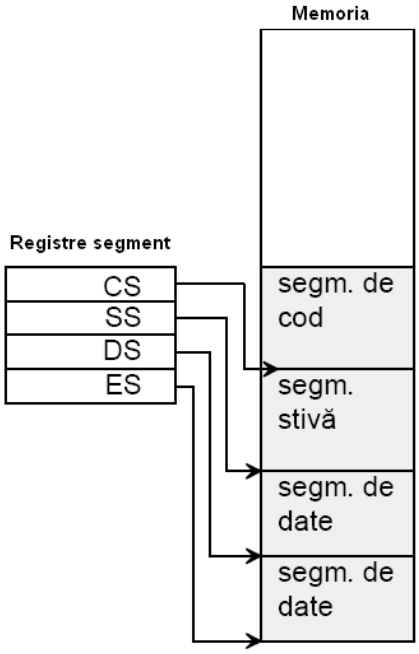
A. Organizarea şi adresarea memoriei

Pentru microprocesorul Intel 8086, memoria este de **1Moctet**.Toate registrele sale interne de adresare fiind de 16 biţi, nu se pot transfera intern adrese mai lungi de 16 ranguri binare, adică nu se pot adresa spaţii de memorie mai mari de 64 kocteti. Pentru acoperirea unui spaţiu de memorie de 16 ori mai mare (64 kocteti x 16 = 1 MB), acesta este divizat logic (în mod virtual) în segmente de maxim 64 kocteti. Aceste segmente de memorie sunt adresabile pe baza unui registru segment. Un segment poate începe de la orice adresă de memorie care este multiplu de 16. Segmentele se pot suprapune, total sau parţial, pot fi adiacente sau disjuncte. **La un moment dat sunt active patru segmente**, **corespunzătoar celor patru registre segment ale lui 8086:**



B. Setul de registre ale microprocesorului 8086

Pot fi clasificate din punctul de vedere al rolului pe care îl au în 5 grupuri:

- registrele generale (pe 16 biţi): **AX, BX, CX, DX.**
- registrele indicatoare de adresă (pe 16 biţi): **SP, BP;**
- registrele index (pe 16 biţi): **SI, DI;**
- registrele segment (pe 16 biţi): **CS, DS, ES, SS;**
- registrul adresei instrucţiunii curente (pe 16 biţi): **IP.**

B1.Registrele generale (AX, BX, CX, DX) - sunt utilizate în instrucţiunile aritmetice şi logice. Toate cele patru registre generale pot fi utilizate şi **ca perechi de registre pe 8 biţi: AX : AH,**

AL, etc.

31		15	8	0	
EAX	.....	AX	AH	AL	Accumulator
EBX	.....	BX	BH	BL	Base reg.
ECX	.....	CX	CH	CL	Counter reg.
EDX	.....	DX	DH	DL	Data reg.

B2.Registrele indicatoare de adresă (SP, BP) - sunt registre care conțin adrese relative în segmentul stivă curent**.** Registrul **SP (Stack Pointer)** are semnificația de adresă curentă a vârfului stivei, iar registrul **BP (Base Pointer)** este utilizat pentru a permite accesul la informația conținută în stivă.

B3.Registrele index (SI, DI) - sunt utilizate în general pentru *adresarea indexată,* conținând adrese relative în segmentul de date curent. Segmentul implicit utilizat în adresare poate să fie modificat prin utilizarea în instrucțiuni a unor prefixe speciale. Registrele index sunt utilizate ca registre index implicite în instrucțiunile de transfer sau prelucrare pe șiruri de octeți: **SI (Source Index) - adresa relativă curentă în şirul sursă, DI (Destination Index) - adresa relativă curentă în şirul destinație.**

31		15	0
ESP	.....	SP	Stack pointer
EBP	.....	BP	Base pointer
ESI	.....	SI	Source index
EDI	.....	DI	Destination index

B4.Registrele segment (CS, DS, ES, SS) - conțin adresa segmentului program (CS), adresa segmentului de date curent (DS), adresa segmentului de date suplimentar (ES) şi adresa segmentului de stivă (SS).

31		15	0
ECS	.....	CS	Code segment
EDS	.....	DS	Data segment
ESS	.....	SS	Stack segment
EES	.....	ES	Extra segment

B5.Registre de stare - sunt utilizaţi pentru a memora informaţii referitoare la rezultatul unor operaţii aritmetice sau logice (AF, CF, OF, PF, SF, ZF) şi pentru memorarea unor informaţii de control pentru microprocesor (DF, IF, TF).

	31	15	0
IP	.....	Adresa instructiune curenta	Instruction pointer
FLAG	.....NT	OF DF IF TF SF ZF AF PF CF	Flag register

CF (Carry) - "1" dacă a apărut un transport din sau s-a făcut un împrumut în rangul cel mai semnificativ. De asemenea instrucţiunile de rotire şi deplasare pot poziţiona acest indicator. **PF (Parity)** - "1" dacă s-a obţinut un rezultat pentru care octetul mai puţin semnificativ are un număr par de biţi cu valoarea "1". **AF (Auxiliar Carry)** - "1" dacă în execuţia unei instrucţiuni care poziţionează acest indicator a apărut un transport din rangul 3 spre rangul 4 sau a fost efectuat un împrumut din rangul 4 spre rangul 3. Acest indicator este utilizat pentru operaţiile cu numere BCD. **ZF (Zero)** - "1" dacă s-a obţinut rezultatul zero. **SF (Sign)** - "1" dacă s-a obţinut un rezultat pentru care bitul cel mai semnificativ este "1". **TF (Trace)** - este utilizat pentru controlul execuţiei instrucţiunilor în regim pas cu pas în scopul depanării programelor. **IF (Interrupt)** - controlează acceptarea semnalelor de întrerupere externă. **DF (Direction)** - indică direcţia de parcurgere a şirurilor de octeţi în cazul instrucţiunilor pe şiruri de octeţi. Valoarea "0" indică parcurgerea de la adrese mici spre adrese mari. **OF (Overflow)** - "1" dacă în execuţia unei instrucţiuni aritmetice cu semn a apărut o depăşire.

C. Programarea inlimbaj de asamblare

1. ***Definirea constantelor:***

```
nume EQU <valoare>
```

unde:

nume - numele constantei. Pentru o constantă nu se rezervă spaţiu în memorie. **<valoare>** - reprezintă valoarea ce este atribuită constantei.

Exemple:

PI equ 3.14159
INT_DOS equ 21h
BASE_ADDR equ 378h

2. ***Declararea datelor:***

```
[nume] <tip> < lista_expresii >

[nume] <tip> < factor >DUP(<lista_expresii>)
```

unde:
[ ] - indică faptul că acest câmp poate lipsi;
[nume] - numele prin care va fi referită data. Acest nume are asociat un tip şi o valoare. Tipul rezultă din tipul datei iar valoarea este adresa la care se va găsi în memorie în timpul execuției programului primul octet rezervat pentru data etichetată cu numele respectiv.
<lista_expresii> - reprezintă lista unor expresii cu ale căror valori se vor inițializa zonele de date rezervate pentru declarația respectivă. Prin **'?'** se specifică faptul că zona respectivă de date nu este inițializată.
<factor> - este un număr care indică de câte ori se repetă lista de expresii care urmează în paranteză.
<tip> - tipul de date:
DB - date de tip octet (BYTE) - 8 biți;
DW - date de tip cuvânt sau adresă (WORD) - 16 biți;
DD - date de tip pointer (DWORD) - 32 biți;
DQ - date de tip constantă reală (QWORD) - 8 octeți;
DT - date de tip constantă BCD (TBYTE) - 10 octeți.

Exemple:

```
x dw 0 ; variabilă de tip întreg
y dw ? ; variabilă de tip întreg ninițializată
ch db 'A' ; variabilă de tip caracter
sir db 'Sir de caractere!' ; şir de caractere
```

3. ***Definirea si utilizarea segmentelor:***

```
<nume> SEGMENT [<tip_aliniere>] [<tip>] [<clasa>]

<corpul_segmentului>

<nume> ENDS
```

unde:
<nume> - este numele asociat segmentului. Acestui nume i se asociază o valoare şi anume adresa de segment corespunzătoare poziției segmentului în memorie în faza de execuție a programului.
Încărcarea unui registru segment cu o adresă de segment se face prin următoarea secvență de instrucțiuni:
MOV AX, <nume>
MOV DS, AX

<tip_aliniere> - poate să fie PARA, BYTE, WORD, PAGE şi indică tipul adresei de început a segmentului, adică adresa de început a zonei de memorie rezervată segmentului este divizibilă cu 16, 1, 2, respectiv 256.
<tip> - este o informație pentru editorul de legături. Această informație indică raportul între acest segment şi segmente definite în alte module obiect.
Având în vedere că într-un program pot să existe mai multe segmente este necesar ca asamblorul să cunoască în fiecare moment care sunt cele patru segmente logice curente pentru a putea să genereze corect codurile instrucțiunilor. De asemenea asamblorul trebuie să ştie ce registru de segment se poate utiliza pentru a realiza accesul la datele referite. Această informație este transmisă asamblorului cu ajutorul pseudoinstrucțiunii **ASSUME**, care are următoarea forma genearală:
ASSUME {<registru_segment> : }

4. ***Definirea si utilizarea macro-urilor:***

```
<nume> MACRO [<listă_parametri>]

<corp_macro>

ENDM
```

<nume> - numele prin care se va identifica macro-ul. Acest nume oriunde apare în codul programului va fi înlocuit cu corpul macro-ului;

5. ***Definirea si utilizarea procedurilor:***

```
<nume> PROC [<atribut>]
<corp_procedură>
<nume> ENDP
```

<nume> - numele prin care se vor face apelurile procedurei. Acest nume are asociată ca valoare adresa primei instrucțiuni din procedură;
<atribut> - **FAR** sau **NEAR**.

6. ***Formatul instrucțiunilor:***

```
[<eticheta>:] [<operație> [<operanzi>]] [;<comentariu>]
```

<eticheta> - un nume care are asociată adresa relativă în cadrul segmentului din care face parte primul octet din instrucțiunea etichetată;

<operaţie> - mnemonica corespunzătoare instrucţiunii;
<operanzi> - este un câmp a cărui existenţă şi formă depind de tipul instrucţiunii.

7. ***Modurile de adresare***

- **Adresare imediată** - în instrucţiune apare valoarea operandului
  mov al, 5
  mov bx, offset nume_variabila
  mov ax, segment_date
- **Adresare la nivel de registru** - operandul referit se găseşte într-un registru.
  mov ax, dx
- **Adresare directă** - în instrucţiune apare adresa operandului
  mov ax, nume_variabila
  mov nume_variabila(2), ax
- **Adresare cu bază** - în instrucţiune apare registrul de bază utilizat. Ca registre de bază se pot folosi registrele BX şi BP.
  mov bx, offset nume_variabila
  mov ax, [bx]
  mov cx, [bx+2]
- **Adresare indexată** - în instrucţiune apare registrul index utilizat.
  mov si, 5
  mov ax, nume_variabila[si]
- **Adresare cu bază şi indexată** - în instrucţiune apar registrele de bază şi index.
  mov ax, nume_variabila[bx][di]
  mov cx, nume_variabila[bx+si]
  mov dx, nume_variabila[bx+si+5]